

Министерство образования Республики Беларусь
Учреждение образования
«Брестский Государственный технический университет»
Кафедра ИИТ

Лабораторная работа №1
По дисциплине «Криптография и сетевая безопасность»
Тема: **«Исследование алгоритмов генерации псевдослучайных последовательностей и методов тестирования чисел на простоту»**

Выполнил:
Студент 4 курса
Группы АС-65
Ярмак К. А.
Проверил:
Хацкевич А. С.

Цель: освоить программную реализацию и практическое применение алгоритмов генерации псевдослучайных последовательностей (РСП), методов их статистической оценки, а также вероятностных и детерминированных тестов простоты чисел с построением доказуемо простых чисел.

Вариант 12

1. Генерация и тестирование псевдослучайных последовательностей (файл "Лабораторная работа №1 (ч.1)")

В рамках данного этапа необходимо разработать программный модуль генерации последовательностей и провести их первичный анализ.

Требования:

- реализовать алгоритм генерации по варианту в виде программной функции/класса, способной выдавать последовательность произвольной длины;
- подобрать параметры генератора (a , c , N , x_0 и др.) так, чтобы добиться максимального периода последовательности T_{max} (согласно теоретическим условиям из методички);
- провести базовое статистическое тестирование полученной последовательности (например, оценить частотность появления элементов, равномерность распределения).

2. Тестирование на простоту и генерация больших простых чисел (файл "Лабораторная работа №1 (ч.2)")

На втором этапе сгенерированные на первом этапе псевдослучайные числа (или базовые механизмы генерации) интегрируются в модуль поиска и проверки чисел на простоту.

Требования:

- реализовать стандартную схему генерации кандидата в простые числа:
 - сгенерировать случайное p -битовое число n (используя генератор из Этапа 1);
 - установить старший и младший биты в 1 (для гарантии разрядности и нечетности);
 - выполнить предварительную фильтрацию (просеивание) делением на малые простые числа (2, 3, 5, 7, ..., вплоть до 256 или 2000), что отсекает до 80% составных чисел без тяжелых тестов;
- интегрировать целевой тест из таблицы вариантов (вместо стандартного Рабина-Миллера или в дополнение к нему) для финальной проверки кандидата. (Примечание: для метода Михалеску используется собственная специфическая схема генерации доказуемо простых чисел).

main.py:

```
import sys
import time

#####
# ч1    lfsr
class LFSR:
    def __init__(self, poly, state):
        self.k = max(poly) # длина регистра = степень
        многочлена
        self.taps = [e for e in poly if e < self.k] # рекуррента s[n+k] = XOR
        s[n+e]
        self.state = state # бит j состояния = s[n+j]

    def bit(self):
        out = self.state & 1
        new = 0
        for e in self.taps:
            new ^= (self.state >> e) & 1
        self.state = (self.state >> 1) | (new << (self.k - 1))
        return out

    def bits(self, n):
        return [self.bit() for _ in range(n)]

    def num(self, n):
        # n-битное число из выхода гены
        v = 0
        for _ in range(n):
            v = (v << 1) | self.bit()
        return v

def period(poly, init):
    # число шагов через которое состояние вернется в init
    g = LFSR(poly, init)
    t = 0
    while True:
        g.bit()
        t += 1
        if g.state == init:
            return t

def chi2(bits, m):
    # хи-квадрат по m-битным блокам алфавит из 2^m слов
    blocks = len(bits) // m
    cnt = [0] * (1 << m)
    for i in range(blocks):
        w = 0
        for b in bits[i * m:(i + 1) * m]:
            w = (w << 1) | b
        cnt[w] += 1
    exp = blocks / (1 << m)
    return sum((c - exp) ** 2 / exp for c in cnt)

POLYS = [
    ("x^5+x^2+1", [5, 2, 0]),
    ("x^5+1", [5, 0]),
    ("x^5+x^4+x^2+1", [5, 4, 2, 0]),
```

```

]
CRIT = {1: 3.841, 2: 7.815, 3: 14.067} # крит значения хи-квадрат alpha=0.05
df=2^m-1

def part1():
    print("многочлен | периоды по всем ненулевым начальным состояниям | при-
митивный")
    for name, p in POLYS:
        k = max(p)
        periods = sorted({period(p, s) for s in range(1, 1 << k)})
        print(f"{name:14} {periods} {'да' if max(periods) == (1 << k) - 1 else
'нет'}")

    print("\nX2 (начальное состояние 1); + равномерность не отвергается, -
отвергается")
    print("крит. значения (alpha=0.05): m=1: 3.84, m=2: 7.81, m=3: 14.07")
    Ns = (1000, 10000, 100000)
    print(f"{'многочлен':14} m " + " ".join(f"N={n:<9}" for n in Ns))
    for name, p in POLYS:
        for m in (1, 2, 3):
            row = []
            for n in Ns:
                x2 = chi2(LFSR(p, 1).bits(n), m)
                row.append(f"{x2:8.1f}{'+' if x2 < CRIT[m] else '-'} ")
            print(f"{name:14} {m} " + " ".join(row))

#####
# ч2    rabin-miller
PRIME_POLY = [19, 5, 2, 1, 0] # x^19+x^5+x^2+x+1 период 2^19-1

def small_primes(limit):
    sieve = [True] * limit
    res = []
    for i in range(2, limit):
        if sieve[i]:
            res.append(i)
            for j in range(i * i, limit, i):
                sieve[j] = False
    return res

def rabin_miller(n, g, rounds=5):
    if n < 4:
        return n in (2, 3)
    if n % 2 == 0:
        return False
    s, t = 0, n - 1
    while t % 2 == 0:
        t //= 2
        s += 1
    for _ in range(rounds):
        a = 2 + g.num(16) % min(n - 3, 1000) # небольшое случайн а из LFSR
        x = pow(a, t, n)
        if x in (1, n - 1):
            continue
        for _ in range(s - 1):
            x = x * x % n
            if x == n - 1:
                break

```

```

    else:
        return False # n составное
return True

```

```

def gen_prime(bits, limit, g):
    sp = small_primes(limit)
    total = sieve_rej = rm_rej = 0
    while True:
        n = g.num(bits) | (1 << (bits - 1)) | 1 # старш и младш бит = 1
        total += 1
        if any(n % q == 0 and n != q for q in sp): # просеивание малыми простыми
            sieve_rej += 1
            continue
        if rabin_miller(n, g):
            return n, total, sieve_rej, rm_rej
        rm_rej += 1

def part2(bits, limit):
    g = LFSR(PRIME_POLY, time.time_ns() % ((1 << 19) - 1) + 1)
    t0 = time.perf_counter()
    n, total, s, r = gen_prime(bits, limit, g)
    dt = time.perf_counter() - t0
    print(n)
    print(f"бит: {bits}, кандидатов: {total}, отсеяно просеиванием (<{limit}): {s},
"
        f"отсеяно Рабином-Миллером: {r}, время: {dt:.4f} c")

```

```

#####
# plots
def plots():
    import matplotlib
    matplotlib.use("Agg")
    import matplotlib.pyplot as plt

    # 1 X2 от длины последовательности для кажд многочлена
    Ns = [100, 300, 1000, 3000, 10000, 30000, 100000]
    fig, axs = plt.subplots(1, 3, figsize=(15, 4))
    for ax, m in zip(axs, (1, 2, 3)):
        for name, p in POLYS:
            bits = LFSR(p, 1).bits(Ns[-1])
            ax.plot(Ns, [chi2(bits[:n], m) for n in Ns], marker="o", label=name)
            ax.axhline(CRIT[m], color="k", linestyle="--", label="критическое (0.05)")
            ax.set_xscale("log")
            ax.set_yscale("log")
            ax.set_title(f"блоки по {m} бит")
            ax.set_xlabel("длина последовательности N")
            ax.set_ylabel("X2")
            ax.legend()
    fig.tight_layout()
    fig.savefig("../tmp/lab1_stage1.png", dpi=120)

    # 2 время и число кандидатов дошедших до рабина миллера от разрядности
    bits_list = [64, 128, 256, 512]
    runs = 200
    fig, axs = plt.subplots(1, 2, figsize=(11, 4))
    for limit in (256, 2000):
        times, rm_cands = [], []
        for b in bits_list:
            t = c = 0

```

```

        for i in range(runs):
            g = LFSR(PRIME_POLY, 1000 * (i + 1) + b)
            t0 = time.perf_counter()
            _, total, s, r = gen_prime(b, limit, g)
            t += time.perf_counter() - t0
            c += total - s # кандидаты прошедшие просеивание
            times.append(t / runs)
            rm_cands.append(c / runs)
        axs[0].plot(bits_list, times, marker="o", label=f"просеивание до {limit}")
        axs[1].plot(bits_list, rm_cands, marker="o", label=f"просеивание до
{limit}")
        axs[0].set_ylabel("среднее время, с")
        axs[1].set_ylabel("кандидатов до Рабина-Миллера, среднее")
        for ax in axs:
            ax.set_xlabel("разрядность p")
            ax.legend()
        fig.tight_layout()
        fig.savefig("../tmp/lab1_stage2.png", dpi=120)

```

```

#####

```

```

# tests

```

```

def check(name, cond):
    print(f"{name}: {'OK' if cond else 'FAIL'}")

```

```

def tests():

```

```

    check("период  $x^{19}+x^5+x^2+x+1 = 2^{19}-1$ ", period(PRIME_POLY, 1) == (1 << 19) -
1)

```

```

    g = LFSR(PRIME_POLY, 1)
    sp = set(small_primes(10000))
    bad = [n for n in range(2, 10000) if rabin_miller(n, g) != (n in sp)]
    check("Рабин-Миллер совпадает с решетом для  $n < 10000$ ", not bad)

```

```

    carm = [561, 1105, 1729, 2465, 2821, 6601, 8911]
    check("числа Кармайкла определяются как составные", not any(rabin_miller(n, g)
for n in carm))

```

```

    check("2^127-1 и 2^521-1 простые", rabin_miller(2**127 - 1, g) and
rabin_miller(2**521 - 1, g))
    check("2^101-1 и 2^128+1 составные", not rabin_miller(2**101 - 1, g) and not
rabin_miller(2**128 + 1, g))

```

```

    n = gen_prime(32, 2000, g)[0]
    ok = n.bit_length() == 32 and all(n % d for d in range(2, int(n ** 0.5) + 1))
    check("сгенерированное 32-битное число простое (проверка делением)", ok)

```

```

if __name__ == "__main__":

```

```

    cmd = sys.argv[1] if len(sys.argv) > 1 else ""
    if cmd == "lfsr":

```

```

        part1()

```

```

    elif cmd == "prime":

```

```

        part2(int(sys.argv[2]) if len(sys.argv) > 2 else 256,
            int(sys.argv[3]) if len(sys.argv) > 3 else 2000)

```

```

    elif cmd == "test":

```

```

        tests()

```

```

    elif cmd == "plot":

```

```

        plots()

```

else:

```
print("Usage: python main.py lfsr | prime [bits] [limit] | test | plot")
```

```
многочлен | периоды по всем ненулевым начальным состояниям | примитивный
x^5+x^2+1      [31] да
x^5+1          [1, 5] нет
x^5+x^4+x^2+1  [1, 15] нет
```

X2 (начальное состояние 1); + равномерность не отвергается, - отвергается
крит. значения (alpha=0.05): m=1: 3.84, m=2: 7.81, m=3: 14.07

многочлен	m	N=1000	N=10000	N=100000
x^5+x^2+1	1	0.8+	10.4-	103.9-
x^5+x^2+1	2	1.3+	15.4-	155.9-
x^5+x^2+1	3	2.7+	24.6-	243.2-
x^5+1	1	360.0-	3600.0-	36000.0-
x^5+1	2	380.0-	3800.0-	38000.0-
x^5+1	3	412.3-	4132.3-	41332.3-
$x^5+x^4+x^2+1$	1	4.4-	44.4-	444.4-
$x^5+x^4+x^2+1$	2	6.7+	66.7-	666.7-
$x^5+x^4+x^2+1$	3	412.3-	4132.3-	41332.3-

```
[kirill@archlinux src]$ python main.py prime 512 256
```

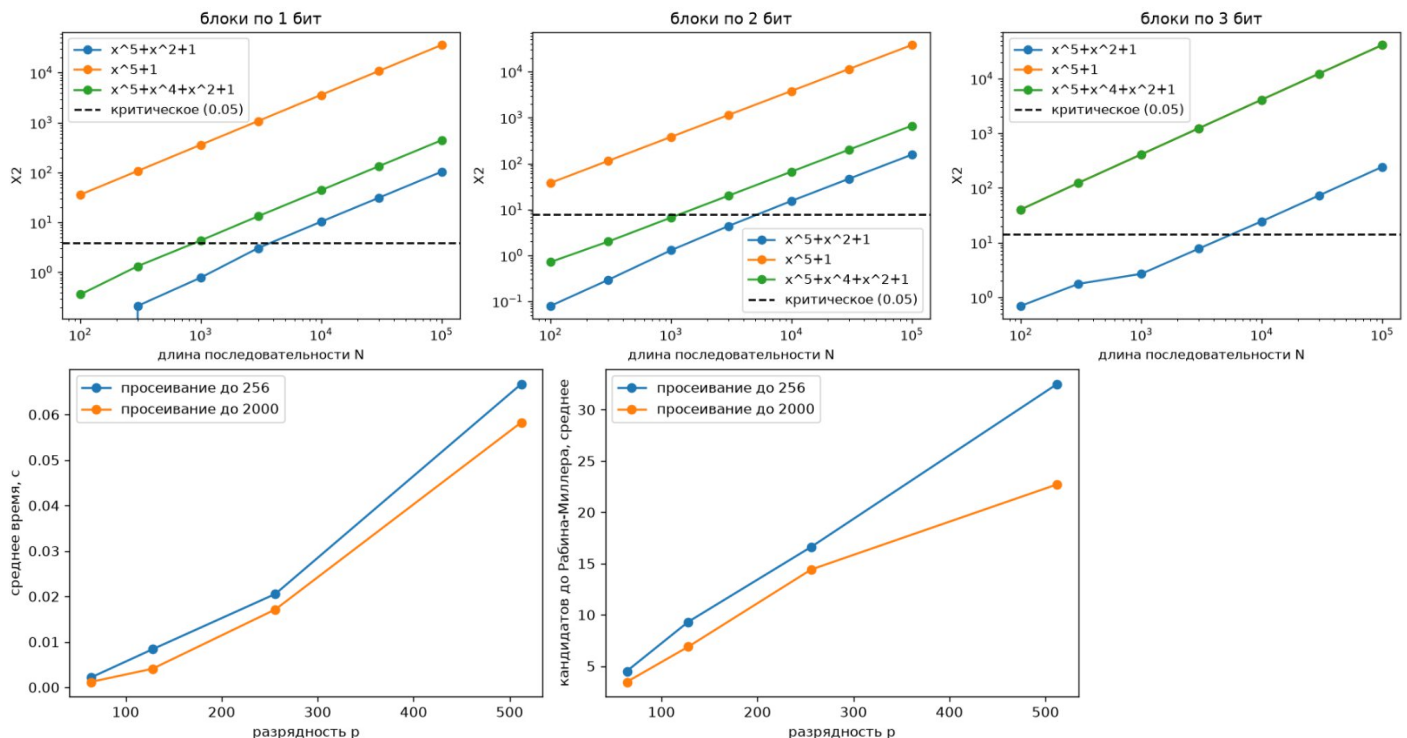
```
13134549886249201477235144337193571494785902894822531927603111558969701208627798323383228079962734498362748127225216568290008706996825159862814886680414179
```

```
бит: 512, кандидатов: 46, отсеяно просеиванием (<256): 35, отсеяно Рабином-Миллером: 10, время: 0.0479 с
```

```
[kirill@archlinux src]$ python main.py prime 512 2000
```

```
9372440024735320272746242111611969055149629130116690074620012569563259519371930337232893609022146068991285970461172499048291223191540009503434376510064963
```

```
бит: 512, кандидатов: 66, отсеяно просеиванием (<2000): 57, отсеяно Рабином-Миллером: 8, время: 0.0583 с
```



Вывод: освоили программную реализацию и практическое применение алгоритмов генерации псевдослучайных последовательностей (РСП), методов их статистической оценки, а также вероятностных и детерминированных тестов простоты чисел с построением доказуемо простых чисел.